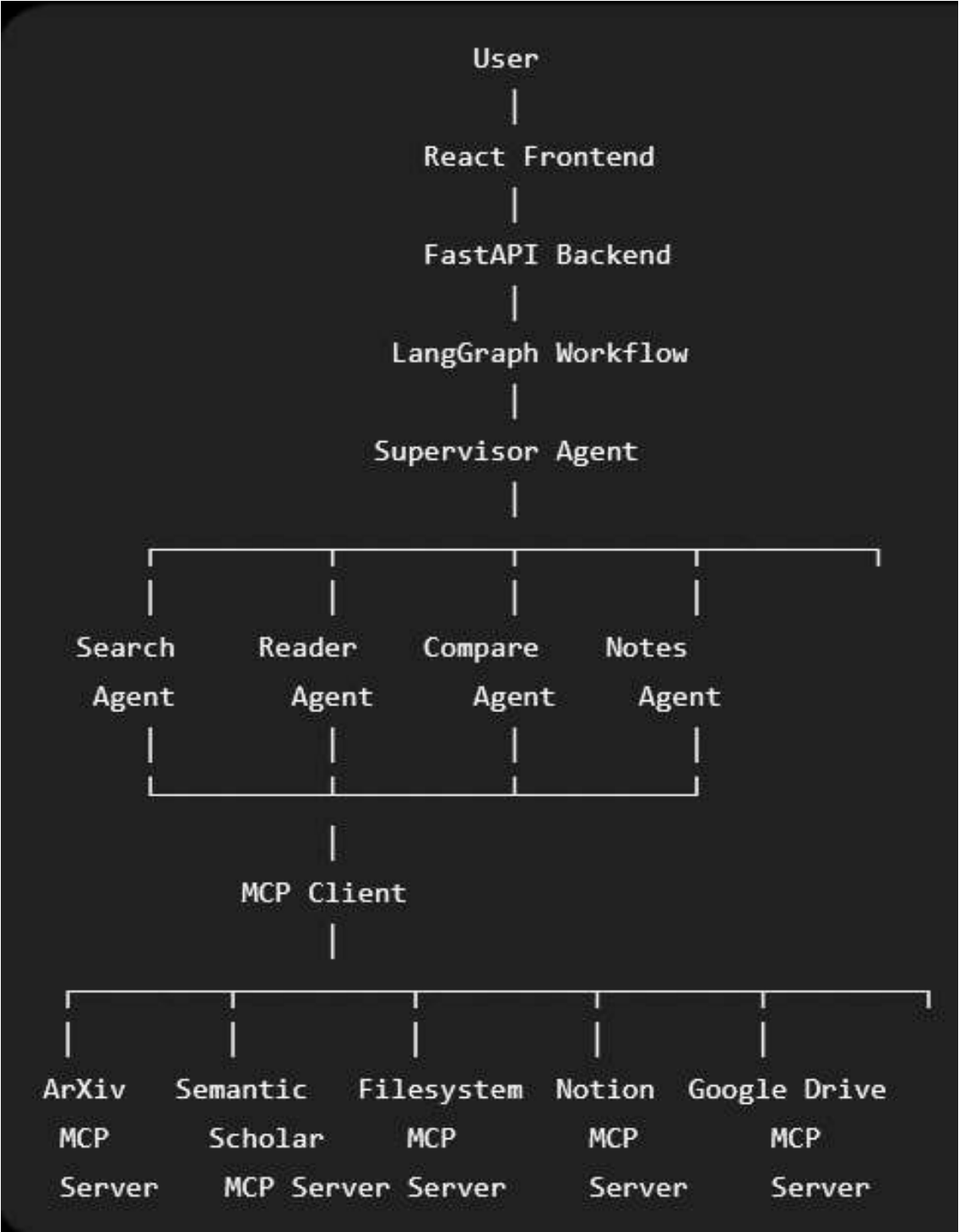




```
AI-Research-Assistant/
|
├── frontend/                # React App
|
├── backend/
|   |
|   ├── main.py              # FastAPI Entry
|   |
|   ├── config.py
|   |
|   ├── agents/
|   |   ├── supervisor.py
|   |   ├── search_agent.py
|   |   ├── reader_agent.py
|   |   ├── compare_agent.py
|   |   └── notes_agent.py
|   |
|   ├── graph/
|   |   └── workflow.py
|   |
|   ├── mcp/
|   |   └── mcp_client.py
|   |
|   ├── tools/
|   |   ├── arxiv.py
|   |   ├── semantic.py
|   |   ├── filesystem.py
|   |   └── notion.py
|   |
|   ├── rag/
|   |   ├── embeddings.py
|   |   ├── vector_store.py
|   |   └── retriever.py
|   |
|   ├── pdf/
|   |   └── parser.py
|   |
|   ├── database/
|   |   └── chromadb/
|   |
|   ├── uploads/
|   |
|   └── requirements.txt
|
└── README.md
```



9. Technologies We'll Use

Technology	Purpose
React	Frontend UI
FastAPI	Backend APIs
LangGraph	Multi-agent orchestration
OpenAI GPT / Ollama	LLM for reasoning and generation
MCP	Standard interface to external tools
ChromaDB	Vector database for RAG
PyMuPDF	PDF text extraction
Sentence Transformers	Generate embeddings
ArXiv MCP	Search research papers
Semantic Scholar MCP	Search research papers
Notion MCP	Save notes
Filesystem MCP	Store summaries locally
Google Drive MCP	Cloud storage

MODULE 1

Project Foundation + ArXiv MCP Search



MODULE 2

PDF Upload + Reader



MODULE 3

RAG + ChromaDB + Paper Q&A



MODULE 4

LangGraph Multi-Agent Workflow



MODULE 5

Paper Comparison + Literature Review + Notes



MODULE 6

Additional MCP Integrations + Final Integration

AI Research Assistant using MCP

Major Project — Modular Implementation Plan

1. Project Architecture

User → React Frontend → FastAPI Backend → LangGraph Workflow → Supervisor Agent → Specialized Agents → MCP Client → MCP Servers

Primary MCP servers: ArXiv, Semantic Scholar, Filesystem, Notion and Google Drive.

2. Module Breakdown

Module	Name	Outcome
1	Foundation + ArXiv MCP Search	Search research papers through ArXiv MCP and display results
2	PDF Upload + Reader	Upload PDFs and extract/read content
3	RAG + ChromaDB + Q&A	Ask questions grounded in uploaded papers
4	LangGraph Multi-Agent System	Supervisor routes requests to specialized agents
5	Comparison + Literature Review + Citations	Compare papers and generate research outputs
6	Full MCP Ecosystem	Add Semantic Scholar, Filesystem, Google Drive and Notion

3. Team Allocation

Module	Person 1	Person 2	Person 3
M1	React Frontend	FastAPI + MCP Client	ArXiv MCP + Search Agent
M2	PDF Upload UI	PDF API + PyMuPDF	Reader Agent
M3	Chat UI	Embeddings + ChromaDB	Retriever + LLM
M4	LangGraph + Supervisor	Search + Reader Agents	Compare + Notes Agents
M5	Compare/Lit Review UI	Comparison Pipeline	Literature Review + Citations
M6	Semantic Scholar MCP	Filesystem + Google Drive MCP	Notion MCP
Final	Frontend Testing	Backend Testing	AI/MCP Testing

4. MODULE 1 — Project Foundation + ArXiv MCP Search

Goal: User enters a research topic → React → FastAPI → MCP Client → ArXiv MCP Server → ArXiv → results → React.

```
Module 1 files:
frontend/
backend/main.py
backend/config.py
backend/agents/search_agent.py
backend/mcp/mcp_client.py
backend/tools/arxiv.py
```

Person 1 — React Frontend

Build SearchPage, SearchBar, PaperCard, PaperList, Loading and ErrorMessage. Show title, authors, publication date, abstract and View Paper. Do not hardcode results.

Person 2 — FastAPI + MCP Client

Create the FastAPI application, GET /api/search?q=..., query validation, CORS and the MCP client connection. Return a stable JSON contract: query + papers[id, title, authors, abstract, published, url].

Person 3 — ArXiv MCP + Search Agent

Set up or implement the ArXiv MCP server integration, expose paper search as an MCP tool, retrieve metadata and normalize results. Keep the full intelligent LangGraph agent architecture for Module 4.

5. MODULE 2 — PDF Upload + Reader

Upload PDF → FastAPI → uploads/ → PyMuPDF → extracted text → Reader Agent.

Person 1: upload UI and status. **Person 2:** POST /api/upload, validation and PyMuPDF. **Person 3:** reader_agent.py and document pipeline.

6. MODULE 3 — RAG + ChromaDB + Paper Q&A;

PDF text → chunks → embeddings → ChromaDB → retriever → LLM → answer.

Person 1: chat UI. **Person 2:** embeddings.py, vector_store.py and ChromaDB. **Person 3:** retriever.py and LLM answer flow.

7. MODULE 4 — LangGraph Multi-Agent System

User → FastAPI → LangGraph → Supervisor → Search / Reader / Compare / Notes Agents → MCP Client.

Person 1: workflow.py + supervisor.py. **Person 2:** Search + Reader agents. **Person 3:** Compare + Notes agents.

8. MODULE 5 — Comparison + Literature Review + Citations

Compare two papers, analyze methodology/datasets/results/limitations, generate literature reviews, and create IEEE/APA/MLA/BibTeX citations.

Person 1: UI. **Person 2:** comparison pipeline. **Person 3:** literature review + citations.

9. MODULE 6 — Full MCP Ecosystem

MCP Client → ArXiv MCP | Semantic Scholar MCP | Filesystem MCP | Notion MCP | Google Drive MCP.

Person 1: Semantic Scholar. **Person 2:** Filesystem + Google Drive. **Person 3:** Notion.